Numpy



© 2023 Olivier Claerbout

Lesmateriaal

- Via GitHub account:
  git clone https://github.com/oclaerbout/syntra_data_scientist.git
- © 2024 Olivier Claerbout

  Dit materiaal mag niet worden gekopieerd, verspreid, gepubliceerd of anderszins gereproduceerd zonder uitdrukkelijke schriftelijke toestemming van de auteur. Dit geldt eveneens voor de Jupyter Notebooks!

Copyright Olivier Claerbout 2024

© 2023 Olivier Claerbout

Doel van de les

- Numpy begrijpen
- Arrays maken met Numpy
- Slicind en Indexing gebruiken
- Numpy Operaties
- Oefening!

© 2023 *Olivier Claerbout*

Intro tot Numpy

© 2023 *Olivier Claerbout*

Copyright Olivier Claerbout 2024

Numpy?

- Veelgebruikte Python Library om met N-dimensionale data arrays te werken en deze te maken
- Numpy is zeer snel, efficiënt en flexibel

© 2023 Olivier Claerbout

Numpy

- Broadcast functies aanwezig
- Lineaire algebra, statistische distributies, trigoniometrie, random,...
- Bijna alle andere libraries die we gaan zien bouwen verder op Numpy!

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Numpy

- Array?
  - Eigenlijk een list
  - Een **Python list** is een soort array:
    - Elementen zijn geïndexeerd
    - Arrays kunnen worden gesliced
    - Editeerbaar (add/remove)
    - Heterogene data
  - Numpy array (ndarray) heeft 2 beperkingen:
    - Vaste grootte na opmaak (mutable maar size niet aanpasbaar)
    - Homogeen

© 2023 Olivier Claerbout

Python list ↔ ndarray

- Beiden hebben indexing, slicing
- Python list:
  - Variabele grootte
  - Heterogeen
  - Elementen zijn Python objecten

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Python list ↔ ndarray

- ndarray:
  - Vaste grootte
  - Eenvoudig reshapen
  - Homogeen
  - Elementen met gespecialiseerde data types
  - Ondersteunt masking: selecteer elementen op basis van conditional statements vb arr[(arr > 2) & (arr < 10)]
  - Fancy indexing: selecteren elementen vb uit 1D array door de indices in een list te plaatsen arr[[0, 3, 4]]

© 2023 Olivier Claerbout

Efficiëntie

- Minder geheugenruimte
- Manipulaties en calculaties zijn veel sneller dan Python
- Dit komt zoals altijd met een kost:
  - Mutable maar Vaste size
  - Homogeen, ook in multidimensionale arrays
  - Datatypes van C dus minder flexibel

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Efficiëntie

- Python gebruikt een variabele lengte voor integers
- Python integers zijn objecten, dus er is veel overhead
- Idem voor floats:
  - Python gebruikt 64 bits om floats op te slaan
  - C heeft ook 32-bit: minder precies, kleinere exponent maar efficiëntere opslag

© 2023 Olivier Claerbout

Numpy Types

- In Numpy kies je zelf je data type
- Vb voor integers:
  - Signed integers int8, int16, int32, int64
  - Unsigned integers uint8, uint16, uint32, uint64
- Vb voor floats:
  - float32, float64 (float64 is compatibel met Python float)
- Zie https://numpy.org/doc/stable/user/basics.types.html

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Grootte van een integer

- Opslag als bits (0/1) sequence
- Het aantal bits bepaalt de grootte van de integer
  - 4 bits: $(1111)_2$ dus $2^0 + 2^1 + 2^2 + 2^3 = 15$
  - Bereik is dus [0, 15] → 16 nummers
  - Indien negatieve nummers: 1 bit voor teken
  - $2^0 + 2^1 + 2^2 = 7$ → -7 -6 -5 -4 -3 -2 -1 -0 +0 +1 +2 +3 +4 +5 +6 +7
  - Nul heeft geen teken nodig dus 1 extra plaats → [-8, 7]

© 2023 Olivier Claerbout

Grootte van een integer

- Signed integers:
  - 8 bits [-128, 127]
  - 16 bits [-32_768, 32_767]
  - 32 bits [-2_147_483_648, 2_147_483_647]
  - 64 bits [-9_223_372_036_854_775_808, 9_223_372_036_854_775_807]
- Unsigned integers:
  - 8 bits [0, 255]
  - 16 bits [0, 65_535]
  - 32 bits [0, 4294967295]
  - 64 bits [0, 18_446_744_073_709_551_615]

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Vectorizatie

- Stel je wil elk element in een array vermenigvuldigen met elk overeenkomstig element in een andere array
  - a = [1, 2, 3, 4]
  - b = [10, 20, 30, 40]
  - Resultaat = [10, 40, 90, 160]
- In Python: via loop of via comprehension [x*y for x, y in zip(a, b)]

© 2023 Olivier Claerbout

Vectorizatie

- Elke iteratie in de loop moet Python een pak werk verrichten, oa type van de te vermenigvuldigen elementen opzoeken (niet homogeen). Homogeen is sneller!
- Daarnaast moet Python proberen de operatie uit te voeren. Dit is niet altijd mogelijk vb string*int kan niet maar Python probeert, ook omgekeerd (vb int*string kan wel!)
- In c met homogene data is dit veel efficiënter, waardoor Numpy veel sneller is

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Vectorizatie

- Numpy sluist ahw de berekeningen door naar C, uitgaand van arrays (ndarray) a en b en een ondersteunde functie of operator zoals add, multiply, divide,...
- Dit heet vectorization: de berekeningen die in Python syntax worden beschreven naar c pushen → veel sneller en efficiënter
- Deze functies zoals add noemt numpy universal functions (ufunc)

© 2023 Olivier Claerbout

Waarom arrays belangrijk zijn

- De meeste data waarmee we werken wordt in arrays voorgesteld, vaak multidimensionaal
  - Afbeeldingen
  - Video's
  - Audio
  - Financiële data
  - Excel data (2D)
  - ...
- Om met grote datasets te werken, moet je weten hoe je efficiënt met arrays kan werken – en daar is Numpy voor!

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Numpy

- NumPy omvat enorm veel universele functies
  - Financieel, wiskunde, statistiek, sorteren, sampling, ...
  - We zullen er enkele bekijken!
- We gaan ook na hoe arrays te maken en manipuleren (indexing, slicing, fancy indexing, masking, reshaping)
- https://numpy.org/doc/stable/
- Pandas en andere libraries zijn gebouwd op NumPy
- Numpy: homogene arrays; Pandas: werken met datasets met ints, floats, strings, booleans, ...

© 2023 Olivier Claerbout

Aanmaken van Arrays
→ Uit een lijst

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Maken van een Array van een list

- Aanmaken van arrays kan:
  - Door een ‘standaard’ list te transformeren
  - Via ingebouwde functies
  - Door genereren van random data

© 2023 Olivier Claerbout

Maken van een Array van een list

- We moeten NumPy importeren, aliased als np
- Het array type is np.ndarray
- Conversie van list naar 1-D array:
  - a = np.array([1, 2, 3])
    - type(a) ➔ ndarray (type van a)
    - Welk type hebben de *elementen* van a?
    - a.dtype ➔ int64

Onthoud: een array is homogeen, dus elk element heet hetzelfde data type. Numpy analyseert de data en kiest een passend type, in dit geval een 64-bit integer. Voor floats is de default 64-bit floats

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

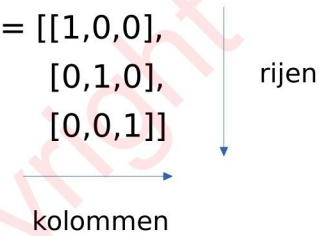
Maken van een Array van een list

- Je kan zelf een data type kiezen ipv Numpy te laten kiezen:
  - a = np.array([1, 2, 3], dtype=np.int8) a.dtype ➔ int8
- Opgelet:
  - Gebruik geen type dat restrictief is
  - Indien een integer te groot is voor het dtype gebeuren vreemde dingen…
  - Floats worden ‘afgekapt’ als dtype = integer
- Waarom niet altijd vb int64? Geheugenefficiëntie bij grote datasets!

© 2023 Olivier Claerbout

Multi-dimensionale Python Lists

- We gebruiken in de les 2 dimensionale arrays:
  - arr_2d = [[1,0,0],
    [0,1,0],
    [0,0,1]]



- Rijen en kolommen worden als “axes” aangeduid:
  - Rijen zijn axis 0
  - Kolommen zijn axis 1

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Multi-dimensionale Python Lists

- NumPy is niet geschikt om echte datasets op te slaan:
  - Stel een dataset die beurskoersen weergeeft
    - Een timestamp (int)
    - Open, high, … : eigenlijk float!

time	open	high	low	close	prev_close
1603249488	100	102	98	102	100
1603249498	200	202	198	202	200
1603249587	300	302	298	302	300

© 2023 Olivier Claerbout

Multi-dimensionale Python Lists

- Conversie multi-dimensionale Python list naar een Array:
  - Werkt identiek als bij 1-D array
  - Opnieuw: alle elementen moeten van hetzelfde data type zijn!

```
list = [
        [1, 0, 0],
        [0, 1, 0],
        [0, 0, 1]
       ]
arr    = np.array(list)                    dtype ➔ int64
arr_8 = np.array(list, dtype=np.uint8)     dtype ➔ uint8
```


© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

De shape van een array

- Shape = het aantal elementen in elke dimensie

[[1, 0, 0], [0, 1, 0], [0, 0, 1]]	→ 2 dimensies met elk 3 elementen	(3,3)
[[1, 2, 3], [4, 5, 6]]	→ 2 dimensies, eerste met 2 el^n, tweede met 3el^n	(2,3)
[1, 2, 3]	→ 1 dimensie, eerste dimensie heeft 3 elementen	(3,)

- De shape krijg je via het shape attribuut van ndarray objecten

© 2023 Olivier Claerbout

Notebook:
01_Numpy_array.ipynb

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Aanmaken van Arrays
→ Van scratch

© 2023 Olivier Claerbout

Arrays from scratch maken

- We hebben gezien hoe arrays van Python lists te maken
- Dit is handig bij data uit bijvoorbeeld een csv file of van een api
- Daarnaast kunnen we ook via specifieke functies arrays maken

Copyright Olivier Claerbout 2024

© 2023 Olivier Claerbout

Nul-Array

- np.zeros(size_of_shape, dtype)
  - size_of_shape:
    - Enkel nummer: 1-D array met lengte ‘nummer’ [0, 0, 0]
    - Tuple: shape (#rijen, #kolommen) [[0, 0, 0],
[0, 0, 0]]
  - dtype: (optioneel)
    - Specifiëren van data type
    - Default: float64

© 2023 Olivier Claerbout

Enkele functies

- np.zeros: arrays gevuld met 0
- np.ones: arrays gevuld met 1
- np.full: arrays gevuld met een gespecifieerde constante waarde
- np.eye: identiteitsmatrices
- np.arange: 1-D array gebaseerd op een range (start-stop-stap)
- np.linspace: gelijkmatig verdeelde nummers tussen start en stop
- np.random.random: arrays gevuld met random floats
- np.random.randint: arrays gevuld met random integers

Copyright Olivier Claerbout 2024

© 2023 Olivier Claerbout

Notebook:
?.ipynb

© 2023 *Olivier Claerbout*

Reshapen van Arrays

© 2023 *Olivier Claerbout*

Copyright Olivier Claerbout 2024

Wat is reshapen?

[1, 2, 3, 4, 5, 6] shape: (6,)

We kunnen deze elementen herschikken:

```
[[1, 2, 3],
 [4, 5, 6]]
```
(2, 3)

```
[[1, 2],
 [3, 4],
 [5, 6]]
```
(3, 2)

```
[[1],
 [2],
 [3],
 [4],
 [5],
 [6]]
```
(6, 1)

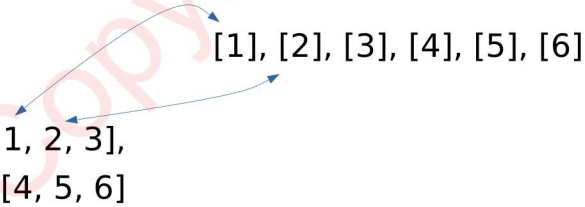
© 2023 Olivier Claerbout

Reshapen: zelfde elementen!

Belangrijk: de herschikte array gebruikt dezelfde elementen als de originele array; er wordt dus geen kopie gemaakt!

Dit geldt eveneens voor slicing (zie later)

```
arr = np.array([1, 2, 3, 4, 5, 6])
reshaped = arr.reshape(3, 2)
```



Wanneer je een waarde wijzigt in de originele of reshaped array, wijzigt deze ook in de andere.

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Kopiëren van een array

- array.copy()
  - Maakt een kopie van array
  - Je kan dit gebruiken om de connectie tussen een array en de reshaped versie ervan te verbreken

© 2023 Olivier Claerbout

Notebook:
?.ipynb

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Stacking

© 2023 Olivier Claerbout

Stacking

- Het concept is eenvoudig:
  - je kan arrays naast elkaar stapelen (hstack)

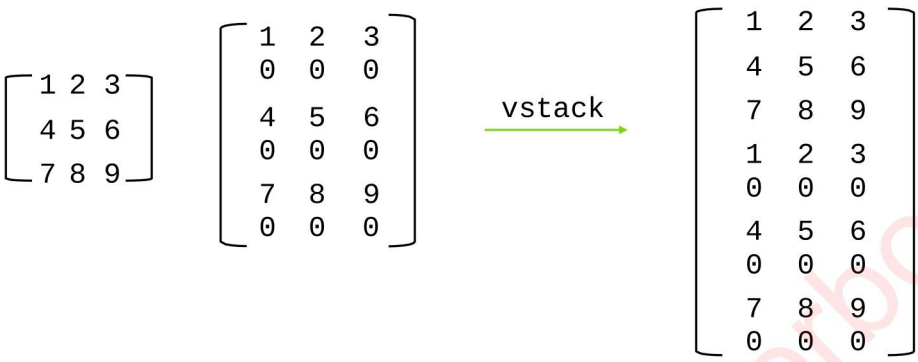


© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Stacking

- Het concept is eenvoudig:
  - en je kan arrays op elkaar stapelen (vstack)



© 2023 Olivier Claerbout

Stacking

- Stel arrays arr_1, arr_2, arr_3
  - np.vstack((arr_1, arr_2, arr_3))
  - np.hstack((arr_1, arr_2, arr_3))

- Argument is een tuple!
- De shapes moeten compatibel zijn om te kunnen stacken!
  - Bij vstack: gelijk aantal kolommen
  - Bij hstack: gelijk aantal rijen

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Stacking

- Wat gebeurt met het dtype bij stacken?
  - Je kan arrays stapelen met verschillend dtype
  - NumPy bepaalt een geschikt gemeenschappelijk gegevenstype
  - Stapelen van uint8, uint16 en int64:
    - NumPy kiest een float64 voor de gestapelde array
      (meest flexibel bij combinatie verschillende gegevenstypes)
  - Vanaf NumPy 1.20 is het mogelijk om het gegevenstype te specificeren bij het gebruik van de functie concatenate - wat een meer generieke vorm is van vstack en hstack

© 2023 Olivier Claerbout

Wijzigen van het Data Type

- We kunnen het data type na stacking controleren door eerst zelf alle te stapelen arrays naar hetzelfde datatype te converteren
- Dit gebeurt met astype:
  - array_1.astype(np.int64)
- Je kan als volgt meerdere arrays stapelen:

```
np.vstack(
        [
        arr1.astype(np.int64),
        arr2.astype(np.int64)
        ]
    )
```


© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Relatie tot ‘originele’ arrays

- We zagen eerder dat een reshaped array volledig gekoppeld is aan de originele array
- Bij gestapelde arrays is dit niet het geval!
  - Het wijzigen van een element in de stack wijzigt de originele array niet
  - Het wijzigen van een element in de originele array wijzigt de stack niet

Stacked arrays zijn niet afhankelijk van de originele arrays!

© 2023 Olivier Claerbout

Notebook:

?.ipynb

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Indexing

© 2023 Olivier Claerbout

Python indexing

- Bij Python sequence types zoals lists en tuples kan je een element opvragen via de index:
  - list = [a, b, c, d, e]
  - list[2] = c
- Vervangen van een element bij mutable types kan eenvoudig:
  - list[0] = f
  - list → [f, b, c, d, e]
- Dit werkt ook bij 2-D lijsten:
  - array = [[1,2], [3,4]]
  - array[1][1] = 4
  - array[0][0] = 5
  - array → [[5,2], [3,4]]

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

NumPy array indexing

- Dit is zeer gelijkaardig als bij Python:
- array = np.arange(1, 7).reshape((2, 3)) → [[1, 2, 3], [4, 5, 6]]
- array[1][1] = 0 → [[1, 2, 3], [4, 0, 6]]

- Bij NumPy arrays kan je ook aan andere manier gebruiken om de index aan te geven, namelijk als tuple: [(i,j)]
  - Bij tuples mogen de haken weg dus [i,j]
  - array[0][0] == array[(0,0)] == array[0,0]

© 2023 Olivier Claerbout

Mutatie van elementen

- Dit is opnieuw zeer gelijkaardig als bij Python:
- array = np.arange(1, 7)
- array[3] = 10 → array ➔ [1, 2, 3, 10, 5, 6]

- array = np.arange(1, 7).reshape((2,3)) → [[1, 2, 3], [4, 5, 6]]

- Array[1,2] = 60 → [[1, 2, 3], [4, 5, 60]]

Opgepast voor data types!

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Notebook:
?.ipynb

© 2023 *Olivier Claerbout*

Slicing

© 2023 *Olivier Claerbout*

Copyright Olivier Claerbout 2023

Slicing van Python sequences

- list = [1, 2, 3, 4, 5]
- list[0:3] → [1,2,3]
- slice_van_list = list[0:3]
- slice_van_list[1] = 7 → slice_van_list → [1,7,3]
  → list → [1, 2, 3, 4, 5]

De slice is onafhankelijk van de originele lijst!

© 2023 Olivier Claerbout

Slicing van Python 2-D sequences

- matrix = [[1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]]

We willen deze vier elementen isoleren, dus slicing over 2 assen

- matrix[0:2] → [[1, 2, 3],
  [4, 5, 6]]
- We kunnen dus de eerste 2 rijen isoleren, maar kunnen niet meer verder om de vier gewenste elementen te isoleren!

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Python sequence slice toewijzingen

- We kunnen een Python list wijzigen via een slice:
  - list = [a, b, c, d, e]
  - list[0:2] = [e, d]
  - list = [e, d, c, d, e]

- Aangezien Python lists geen vast aantal elementen hebben, kunnen we er meer of minder toevoegen bij vervanging via slicing:
  - list = [a, b, c, d, e]
  - List[0:2] = [0, 3.14, e, d]
  - List = [0, 3.14, e, d, c, d, e]

Dit werkt omdat Python lists niet vast zijn, ze kunnen worden gewijzigd – ze zijn mutable en heterogeen!

© 2023 Olivier Claerbout

Slicing van 1-D NumPy Arrays

- Opnieuw zeer gelijkaardig als bij Python:
  - array = np.array([0,1,2,3,4,5,6,7,8,9,10]) → array[0:2] → [0,1]

ndarray, geen list!

  - Ook step indexing, negative indexing etc zijn ondersteund, evenals list slicing
  - Step indexing of slicing algemeen: lijst[start : stop : step]
    - Omgekeerd → [::-1]
    - Elk nde element → [::n]
    - Dus array[2:5:2] → [2,4]

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Slicing van 2-D NumPy Arrays

- Numpy ondersteunt slicing langs meerdere assen itt Python!

```
[ [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9] ]
```

Langs axis 0: slice 0:2

Langs axis 1: slice 1:3

→ array[0:2, 1:3] of array[:2, 1:]

- We kunnen ook steps gebruiken: hoe krijgen we volgende:

```
1   2   3   4   5
6   7   8   9   10
11  12  13  14  15
16  17  18  19  20
21  22  23  24  25
```

Langs axis 0: rijen 0,2,4 dus [::2]

Langs axis 1: rijen 1,3 dus [1::2]

→ array[::2, 1::2]

© 2023 Olivier Claerbout

Slice toewijzing in NumPy Arrays

- Gelijkaardig als bij list slices
  - Je kan niet vervangen door een array die niet dezelfde shape heeft!
  - Je kan de shape van de originele array (Numpy arrays: fixed size!)
  - Ook hier: oppassen met data types!

```
a = np.array([1, 2, 3, 4, 5])
a[0:3] = np.array([10, 20, 30])   a → [10, 20, 30, 4, 5]
```

Je kan ook vervangen met een list of tuple – NumPy kan dit verwerken

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Slice toewijzing in NumPy Arrays

- Je mag ook één enkele waarde aan een slice toewijzen
- Numpy zal de volledige slice ‘vullen’ met die waarde
- Dit heet ‘broadcasting’

arr = np.array([1, 2, 3, 4, 5, 6, 7])

arr[::3] ➔ [1, 4, 7]
arr[::3] = 0 arr ➔ [0, 2, 3, 0, 5, 6, 0]

© 2023 Olivier Claerbout

Slices zijn “gekoppeld” aan de originele array

- Zie ook reshape
- Een slice is gekoppeld aan de array waarvan de slice afstamt

arr = np.array([1, 2, 3, 4, 5])
s = arr[0:3] ➔ [1, 2, 3]

- Het vervangen van een element in s zal impact hebben in arr en omgekeerd
- Dit kan vermeden worden door een kopie van de slice te maken

s = arr[0:3].copy()

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Notebook:
?.ipynb

© 2023 *Olivier Claerbout*

Fancy Indexing

© 2023 *Olivier Claerbout*

Copyright Olivier Claerbout 2023

Fancy indexing

- We hebben gezien hoe we kunnen items in een array selecteren
  - Via enkelvoudige index waarden vb arr[3] voor 1-D of arr[2,5] voor 2-D
- We hebben slicing gezien
  - Voor 1-D: arr[1:3:2]
  - Voor 2-D: arr[1:3:2, :5]
- Wat als we rijen 1, 2 en 4 willen?

© 2023 Olivier Claerbout

Eén manier...

arr = np.array([1, 2, 3, 4, 5, 6])

Stel: we willen een array met elementen met index 0, 1, 3 en 5
→ sub = np.array([arr[0], arr[1], arr[3], arr[5])

Dit werkt!
Maar… wat we hier eigenlijk hebben is een array met indices!

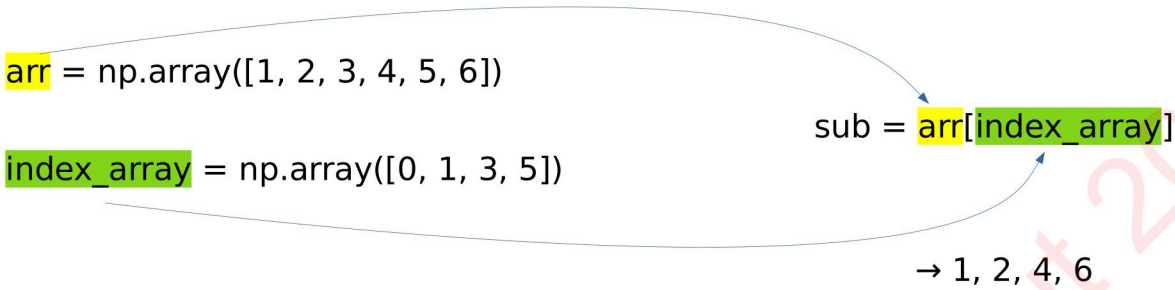
→ NumPy ondersteunt het specifiëren van elementen met behulp van een *array met indices* in plaats van één enkele index!

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Fancy Indexing

Gebruik een array met indices (een index array)



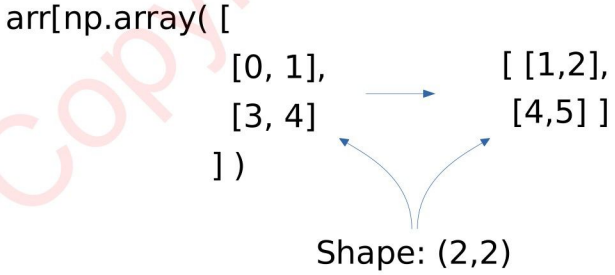
Je kan de index array ook inline meegeven:
sub = arr[np.array([0, 1, 3, 5])]

© 2023 Olivier Claerbout

Shape van de index array

De shape van de index array bepaalt de shape van de selectie:

arr = np.array([1, 2, 3, 4, 5, 6])
arr[np.array([0, 1, 3, 4])] ➔ [1, 2, 4, 5]



© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Fancy indexing in meerdere dimensies

Fancy indexing kan worden toegepast op meerdere assen:

as 1 as 2

[index_array, index] Of omgekeerd: [index, index_array]

[index_array, slice] Of omgekeerd: [slice, index_array]

[index_array, index_array]

Enkele voorbeelden...

© 2023 Olivier Claerbout

[index_array, index]

Fancy indexing kan worden toegepast op meerdere assen:

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

arr[1, np.array([0,1,3])]

→ enkele rij → [6,7,9]

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

arr[np.array([0,1,3]), 1]

→ enkele kolom → [2,7,17]

De resulterende arrays *zijn 1-D!*

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

index_array met slice

Fancy indexing kan worden toegepast op meerdere assen:

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

arr[1:3, np.array([0,1,3])]

↳ Meerdere rijen, meerdere kolommen

→ [[6, 7, 9],
[11,12,14]]

arr[:, np.array([0,3])]

↳ Alle rijen, kolommen 0 en 3

© 2023 Olivier Claerbout

index_array met index_array

Dit wordt niet vaak gedaan omdat het verwarrend kan zijn…

Zorg dat je index arrays dezelfde shape hebben om het enigszins overzichtelijk te houden

1-D en 1-D index array:

arr[np.array([0, 2]), np.array([1, 3])]

→ denk aan het zippen van de indices van beide assen:

→ (0,1)(2,3)

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

→ [2,14]

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

index_array met index_array

2-D en 2-D index array:

→ opnieuw zippen van de indices van beide assen

→ nu is onze “index array” ook 2-D

arr [np.array ([[0, 1], [3,4]]) ,
np.array ([[0, 2], [1,3]])]

We doen fancy indexing met een 2-D array in de eerste as, en een 2-D array in de tweede as!

Index array 1: 0 1 / 3 4

Index array 2: 0 2 / 1 3

→ ‘ge-zipt’: (0,0) (1,2) / (3,1) (4,3)

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

[[1, 8],
[17,24]]

Fancy indexing op beide assen: niet vaak gebruikt!

© 2023 Olivier Claerbout

Notebook:

?.ipynb

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Masking

© 2023 Olivier Claerbout

Boolean Masking

Bij Boolean Masking wordt een expressie toegepast die als resultaat een boolean value oplevert voor elk element van een array

Hierdoor ontstaat een array met True/False, met dezelfde shape

Deze array wordt gebruikt om de originele array te ‘filteren’: de True waarden blijven behouden, de False waarden niet.

	%2 == 0	Na filter:
1	False	
2	True	2
3	False	
4	True	4
5	False	

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Comparison Functions

Dit zijn functies die kunnen worden toegepast op elk element van een array.
Ze retourneren een array met het resultaat voor elk element

Voorbeeld: np.less(arr, value)
→ kijkt naar elk element van arr en evalueert of element < value

arr = np.array([1, 2, 3, 4, 5])
np.less(arr, 4) ➔ [True, True, True, False, False]

© 2023 Olivier Claerbout

Logische Functies in NumPy

greater less_equal not_equal equal etc…
→ zie https://numpy.org/doc/stable/reference/routines.logic.html

We kunnen ook gewoon vergelijkende operator symbolen gebruiken:
< <= > >= == !=

NumPy zal de overeenkomstige functies gebruiken.

Copyright Olivier Claerbout 2024

© 2023 Olivier Claerbout

Toepassen van een masker

- Een array met True / False waarden noemen we een mask
- We kunnen dit toepassen op een array (van dezelfde shape!)

```
arr = np.array([1, 2, 3, 4])

mask = np.array([True, True, False, True])
```

→ Je kan ook gewoon volgende gebruiken: `mask = arr != 3`

`arr[mask]` ➔ [1, 2, 4]

Dit kan allemaal in een enkelvoudig statement: `arr[arr != 3]`

© 2023 Olivier Claerbout

2-D arrays maskeren

- De toepassing van een mask geeft altijd een 1-D array terug, zelfs bij maskeren van een 2-D array
- De reden hiervoor is eenvoudig: NumPy kan onmogelijk weten welke en hoeveel waarden terugkomen (de True values) vb per rij.
- Daarom is het resultaat altijd 1-D

```
arr = np.array( [ [1, 2],
                  [3, 4] ] )   mask = arr != 3 →  Mask: [ [True,   True],
                                                          [False, True] ]
```

`arr[arr != 3]` ➔ [1, 2, 4] → 1-D resultaat!

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Combineren van logische operatoren

- Python gebruikt: **and** **or** **not**
- NumPy gebruikt: **&** **|** **~**

- Omwille van voorrangsregels gebruik je best haakjes ( ) om logische expressies te groeperen

arr = np.arange(-10, 10)

arr[(arr > 0) & (arr % 2 == 0)] → [2, 4, 6, 8]

© 2023 Olivier Claerbout

Notebook:

?.ipynb

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Universele functies

© 2023 Olivier Claerbout

Universele Functies

- Universele functies zijn gevectoriseerde functies
  - Ze passen een functie toe op elk element in een array
  - Het ‘zware werk’ gebeurt in C, en niet in Python
  - Hierdoor is dit zeer snel
  - Numpy heeft zeer veel universele functies – wellicht zit wat je zoekt erin (https://numpy.org/doc/stable/reference/ufuncs.html#available-ufuncs)
    - Math operations (logs, exponentials, sqrt, abs,…)
    - Trigoniometrisch en Hyperbolisch
    - Vergelijkende (equal, less than, greater than, min/max, …)
  - Je kan zelf ook een functie schrijven die gebruik maakt van vectorization (buiten de scope)

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Universele Functies en operatoren

- add, subtract, multiply, divide, floor_divide, mod, power, …
- Kunnen als functies worden aangeroepen, waarbij minstens één van de argumenten een array moet zijn
  - np.add(arr_1, arr_2)
  - np.add(arr_1, scalar)

- In plaats van add kan je ook de **+** operator gebruiken:
  - Python zal in de achtergrond np.add gebruiken
  - Dit geldt ook voor -, *, /, //, %, **

© 2023 Olivier Claerbout

Universele Functies en operatoren

- Element-wise optelling: [1, 2, 3] + [4, 5, 6] = [5, 7, 9]
- Element-wise modulo operatie: [1, 2, 3] % [4, 5, 6] = [1, 2, 3]
- Element-wise machtsverheffing:

```
[[1, 2, 3],
 [4, 5, 6]] ** [[1, 2, 1],
               [2, 1, 2]] = [[ 1,   4,  3  ],
                             [16,  5, 36 ]]
```


© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Universele Functies en operatoren tussen arrays

$[a_0, a_1, a_2] + [b_0, b_1, b_2] \rightarrow [a_0 + b_0, a_1 + b_1, a_2 + b_2]$

$[a_0, a_1, a_2] \,\%\, [b_0, b_1, b_2] \rightarrow [a_0 \,\%\, b_0, a_1 \,\%\, b_1, a_2 \,\%\, b_2]$

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{bmatrix} ** \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \end{bmatrix} \rightarrow \begin{bmatrix} a_{00} ** b_{00} & a_{01} ** b_{01} & a_{02} ** b_{02} \\ a_{10} ** b_{10} & a_{11} ** b_{11} & a_{12} ** b_{12} \end{bmatrix}$$

Arrays behouden hun vorm, maar het is technisch gezien mogelijk om dankzij broadcacting verschillende vormen te gebruiken

(https://numpy.org/doc/stable/user/basics.broadcasting.html)

© 2023 Olivier Claerbout

Universele Functies & operatoren tussen array & scalar

De eenvoudigste vorm van broadcasting:

$[a_0, a_1, a_2] * 3 \rightarrow [a_0, a_1, a_2] * [3, 3, 3]$

$$1 \,/\, \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \,/\, \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{bmatrix}$$

Broadcasting zorgt voor een overeenkomende shape.

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Wanneer shapes niet overeenkomen

Soms kan je bij niet overeenkomende shapes toch de bewerking doorvoeren via broadcasting:

```
a1 =  [ [1,2,3],          a2 = [10, 20, 30]
        [4,5,6],
        [7,8,9] ]
```

*a1*a2 is mogelijk door a2 drie keer te herhalen zodat de shape van a2 overeenkomt met de shape van a1.*

© 2023 Olivier Claerbout

Notebook:

?.ipynb

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Extra: Math & Stats Functions

© 2023 Olivier Claerbout

Bijkomende functies in NumPy

NumPy heeft bijzonder veel functies om arrays te bewerken en wiskundige bewerkingen uit te voeren, onder meer:

- Trigoniometrie
- Hyperbolische functies
- Logs
- Exponenten
- Lineaire algebra (matrix/vector products, eigenfuncties/values,…)
- Statistiek (gemiddelde, variantie, correlatie, histogram)
- …

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Gespecialiseerde libraries

Daarnaast zijn er nog talloze gespecialiseerde libraries gebouwd bovenop NumPy en Pandas:

- SciPy (optimalizatie, integratie,…)
- Statsmodels (regressie, time series, …)
- Pyfolio (portfolio performance en risk analysis)
- QuantLib (financieel)
- Quandl (financiële datasets)

© 2023 Olivier Claerbout

Bewerkingen over assen

- We hadden het eerder over rijen (axis 0) en kolommen (axis 1)
- Veel universele functies in NumPy kunnen berekeningen doen
  - Op de array als geheel
  - Langs een as

Voorbeeld: de amax functie om de maximale waarde te vinden in een gegeven array of langs een as van de array:

```
a1 = [ [1,2,3],        np.amax(a1) = 9
       [4,5,6],
       [7,8,9] ]
```

→ Werkt over alle elementen van een array!

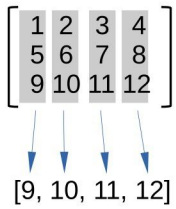
© 2023 Olivier Claerbout

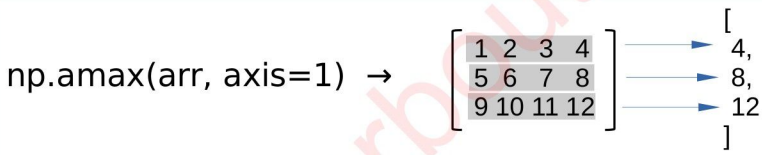
Copyright Olivier Claerbout 2024

Max

- We kunnen ook aangeven over welke as de functie moet werken:

np.amax(arr, axis=0)
→ het maximum wordt gezocht over de rijen heen – dus per kolom!





© 2023 Olivier Claerbout

Max

- Sommige functies kunnen enkel element per element werken:
  - sin, sinh, log, exp, …
- Sommige functies die op groepen data werken zoals amax ondersteunen het werken over assen heen
  - amax, amin, mean, median, std, sum, cumsum, product,…

https://numpy.org/doc/stable/reference/routines.math.html

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

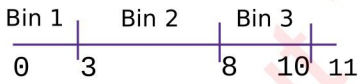
Histogram

- np.histogram maakt een frequentie distributie met bins aan

stel a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

We hebben dus volgende bins:

- Bin 1: 3 elementen van 0 tot maar exclusief 3
- Bin 2: 5 elementen van 3 tot maar exclusief 8
- Bin 3: 3 elementen van 8 tot maar exclusief 11



bins → [0, 3) [3, 8) [8, 11)

We kunnen de bins definiëren als een array waarin de linkerrand wordt meegegeven, en de uiterst rechtse rand (die inclusief is): bins = [0,3, 8, 10]

© 2023 Olivier Claerbout

Histogram

- Om een histogram te maken geef je de array en de bins array mee: np.histogram(a, bins_arr)
  - Dit retourneert een tuple: (array frequencies, bins array)

3, 5, 3

  - De bins array wordt geretourneerd omdat je soms gewoon een integer geeft voor het aantal gewenste bins. De functie berekent dan gelijk verdeelde bins gebaseerd op het minimum en maximum van de array.
  - Je kan nog extra argumenten meegeven
    (zie https://numpy.org/doc/stable/reference/generated/numpy.histogram.html)

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024